

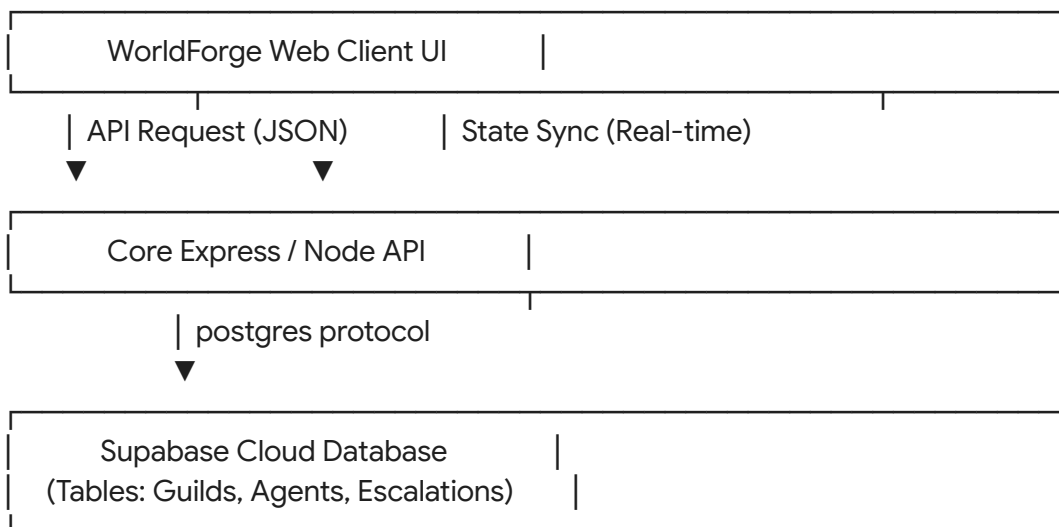
# System Blueprint: Phase 1 Agent Foundry

*Version 1.0 — Architecture Blueprint for Core Service Engine*

This specification defines the baseline architecture for the WorldForge Agent engine. It provides the foundation for agent generation, database integration, and UI state synchronization.

## 1. System Overview

The Foundry Engine acts as the bridge between the local frontend workspace and our cloud services (Supabase DB). It manages the definitions of user-configured guilds, roster agents, and state caching.



## 2. Database Schema (Supabase PostgreSQL)

These SQL definitions must be pasted directly into your Supabase SQL Editor GUI dashboard.

```
-- Create Guilds Table
CREATE TABLE guilds (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
```

```
);

-- Create Agents Table
CREATE TABLE agents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  guild_id UUID REFERENCES guilds(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  responsibility TEXT NOT NULL,
  authority TEXT NOT NULL,
  memory_touchpoints TEXT,
  escalation_targets TEXT[] DEFAULT '{}':TEXT[],
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc':text, now()) NOT NULL
);

-- Insert Seed Data
INSERT INTO guilds (key, name, description)
VALUES ('core', 'Core Production', 'Primary execution guild for WorldForge processes');
```

### 3. Implementation Code Files

Claude must generate these core backend files inside the workspace to handle the service routing.

#### A. Environment Configuration (.env.example)

```
PORT=5000
SUPABASE_URL=your-supabase-project-url
SUPABASE_ANON_KEY=your-supabase-anonymous-key
```

#### B. Express Server Entrypoint (src/server.js)

```
const express = require('express');
const cors = require('cors');
const { createClient } = require('@supabase/supabase-js');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

const supabase = createClient(process.env.SUPABASE_URL,
process.env.SUPABASE_ANON_KEY);
```

```
const PORT = process.env.PORT || 5000;
```

```
// Get full active roster data
```

```
app.get('/api/roster', async (req, res) => {
```

```
  try {
```

```
    const { data: guilds, error: gError } = await supabase.from('guilds').select('*');
```

```
    if (gError) throw gError;
```

```
    const { data: agents, error: aError } = await supabase.from('agents').select('*');
```

```
    if (aError) throw aError;
```

```
// Structure into unified WorldForge JSON format
```

```
const structured = guilds.map(g => ({
```

```
  key: g.key,
```

```
  name: g.name,
```

```
  agents: agents.filter(a => a.guild_id === g.id).map(a => ({
```

```
    name: a.name,
```

```
    resp: a.responsibility,
```

```
    auth: a.authority,
```

```
    escalatesTo: a.escalation_targets,
```

```
    mem: a.memory_touchpoints
```

```
  )))
```

```
  ));
```

```
  res.json({ rosterVersion: "5.2.0", guilds: structured });
```

```
  } catch (err) {
```

```
    res.status(500).json({ error: err.message });
```

```
  }
```

```
});
```

```
// Update agent schema
```

```
app.post('/api/agents', async (req, res) => {
```

```
  const { guild_key, name, resp, auth, escalatesTo, mem } = req.body;
```

```
  try {
```

```
    const { data: guild, error: gErr } = await supabase.from('guilds').select('id').eq('key',
```

```
    guild_key).single();
```

```
    if (gErr) throw gErr;
```

```
    const { data, error } = await supabase.from('agents').insert([
```

```
      {
```

```
        guild_id: guild.id,
```

```
        name,
```

```
        responsibility: resp,
```

```
    authority: auth,
    escalation_targets: escalatesTo,
    memory_touchpoints: mem
  });
  if (error) throw error;

  res.status(201).json({ success: true, data });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

app.listen(PORT, () => {
  console.log(`WorldForge Service Engine active on port ${PORT}`);
});
```

## 4. Next Steps for Claude (Automated Execution Prompt)

"""Claude, read /docs/specs/blueprint\_v1\_agent\_foundry.md. Initialize our backend codebase using the filesystem tool:

1. Create the src directory if it does not exist.
2. Generate .env.example using the schema in the blueprint.
3. Write src/server.js with the full operational Express service code.
4. Set up a basic package.json file including the dependencies: express, cors, @supabase/supabase-js, and dotenv."""